LINE OA Plus TW - DX Solution Technical Document (v2)

Version	Date	Description
v2.0.0	2026-4-27	1. Support Development mode on OA plus 2. Support Mobile Device 3. Support more user info from sdk & 4. dx-jwt 5. Open API: new feature 6. Webhook

Introduction

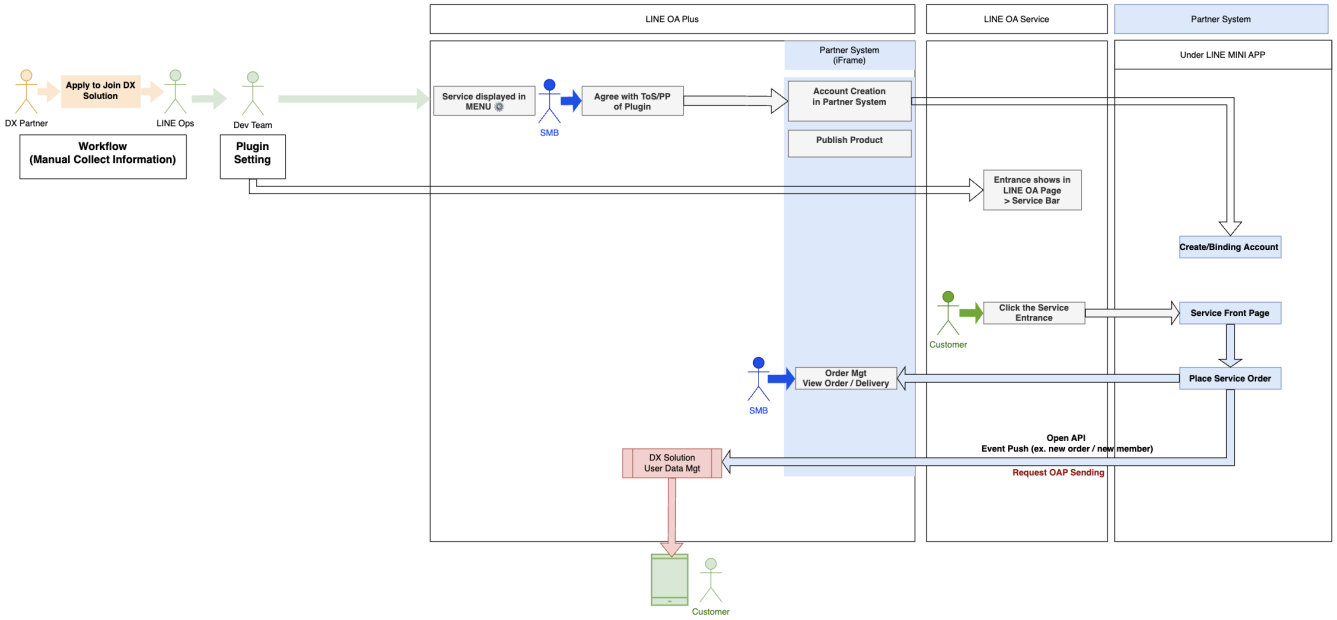
The purpose of this document is to provide a comprehensive guide for DX Solution (a.k.a Digital Experience Solution) with the LINE OA Plus TW system, specifically designed for LINE Tech Partners. This integration aims to create a seamless experience that leverages the full potential of LINE's capabilities, thereby enhancing service delivery and operational efficiency for businesses.

By facilitating this integration, LINE Tech Partners can offer solutions that are tightly coupled with LINE's robust platform, enabling businesses to manage their operations more effectively. This document outlines the necessary requirements, specifications, and processes to ensure a smooth and successful integration, empowering partners to deliver enhanced value to their clients.

System Overview

The integration with LINE OA Plus for DX Solution Tech Partners requires the development and deployment of two key applications. These applications are designed to enhance the functionality and user experience for both merchants and customers within the LINE ecosystem.

> The document is currently only effective in the Staging environment. The availability date for Production will be provided in a future release.



What LINE's Tech Partner need to develop

Admin Plugin for LINE OA Plus TW

The application is an admin plugin embedded within the LINE OA Plus TW platform. This plugin is specifically targeted at small to medium-sized businesses, providing them with the tools to manage service transactions and configurations efficiently through a backend interface. Key features of this admin plugin include:

- **Service Management**: Businesses can oversee and configure their service offerings, ensuring seamless transaction handling.

MINI App for LINE Customers

The application is an unverified MINI App designed for LINE customers operating within the OA. This app leverages LINE Login to access user information, facilitating a direct and integrated user experience. Key functionalities of the MINI App include:

- **User Interaction Services**: Customers can interact with various services such as orders, membership, reservations, and coupons directly within LINE.
- **Event Message**: The app can send messages on behalf of the OA to users, ensuring timely communication and engagement.

Together, these applications harness the power of LINE's platform, enabling businesses to enhance their operational efficiency and provide enriched services to their customers. This system overview highlights the strategic integration points and core functionalities necessary for a successful partnership with LINE Tech Partners.

For Tech Partner unverified MINI Apps, it is required to provide corresponding environments to ensure proper development, testing, and production deployment:

- **Production**:

  The live environment used by end customers.
- **Staging**:

  Used for QA and validation before production release.

  - The **channel status** must be switched to **Developing** to prevent external access.
  - LINE QA tester accounts must be added as **Testers**.
  - Reference guide: How to test login channel.

By separating environments and ensuring correct channel configurations, Tech Partners can safely validate new features, protect production users, and streamline the release process.

What component LINE provide for integration

LINE OA Plus TW SDK

The LINE OA Plus TW SDK is designed to enable seamless integration with the LINE ecosystem when embedded within the OA Plus TW platform. It provides access to various LINE services, enhancing the capabilities of the Admin Plugin from the front-end perspective.

Open API

Send Event API:

The Send Event API facilitates seamless integration for partners developing an Unverified MINI App for LINE Customers. This API enables event synchronization and message dispatch without the need to obtain the OA account's Channel ID and Channel Secret. Partners can leverage the SMB OA to send messages directly, enhancing communication and engagement with users.

Key Features:

- **Event Synchronization:** Effortlessly sync events to ensure up-to-date information and interactions within the MINI App.
- **Message Dispatch:** Send timely messages on behalf of the OA, utilizing LINE's robust platform for customer engagement.

This API empowers partners to integrate essential functionalities, enhancing the user experience and operational capabilities within the LINE ecosystem.

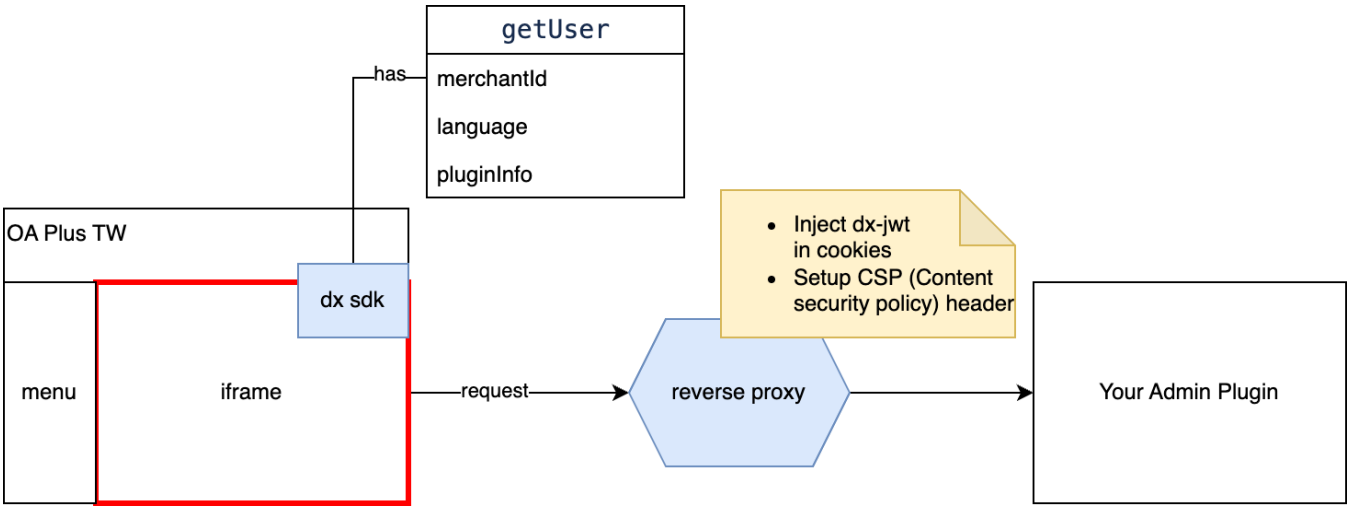
Other Open APIs:

- **Retrieve Payment Detail**: After the payment completion notification, the Tech Partner can query detailed payment information to issue invoices.
- **Retrieve OA Subscription Status**: After the subscription status change notification, the Tech Partner can query the OA's subscription status to sync it with OA Plus.

Partner Webhook Notifications

The Partner Webhook allows LINE OA Plus to proactively notify Tech Partners when key events occur within the platform, such as subscription status changes or payment completions. When an event is triggered, OA Plus sends an HTTP POST request to the partner's pre-registered webhook URL, containing a signed JSON payload.

Admin Plugin Integration with LINE OA Plus TW



Prerequisites

Technical Environment

- **Supported Environments**:
  - **Staging Environment**: This environment is used for testing and development purposes. It should mimic the production environment as closely as possible to ensure accurate testing and validation.
  - **Production Environment**: This is the live environment where the integration will be deployed for end-users. It must be stable, secure, and optimized for performance.
- **System Requirements**:
  - Ensure that both the staging and production environments are set up with the necessary hardware and software configurations to support the integration.
  - Maintain consistent versions of operating systems, databases, and any third-party software across both environments to avoid compatibility issues

Account and Access Requirements

- **Access Credentials**:
  - Obtain the necessary biz account and access credentials for both staging and production environments.
  - Ensure that credentials are securely stored and managed, following best practices for security.
- **Permissions**:

- Verify that all required permissions and roles are assigned for accessing and managing the integration components in both environments.

- **DX-JWT Signature Verification**:
  - Each request to the admin plugin must include a JWT.
  - The DX-JWT signature must be verified using the secret provided by us.
  - This ensures that the request is authentic and has not been tampered with.
  - Structure
    - Header
      - ```
        {
         "alg": "HS256",
         "typ": "JWT"
        }
        ```
    - Payload
      - ```
        {
         "pluginId": "12345",
         "merchantId": "123456",
         "pluginInfo": {
            "level": "FREE",
            "activatedStatus": "ACTIVATED"
         },
         "iat": 1717023000,
         "exp": 1717026600
          ...
        }
        ```
    - Signature
      - ```
        HMACSHA256(
         base64UrlEncode(header) + "." +
         base64UrlEncode(payload),
         Secret of Admin Plugin)
        ```

Admin plugin API integration

- **Integration requirement**
  - API requests must be sent through the reverse proxy for authentication and request forwarding.
- **Domain requirement**
  - API requests must be made to the same plugin domain that serves the embedded plugin page.
- **Example**
  - Plugin URL: `https://testplugin-dx-staging.line-apps.com/`
        ```

- API request URL: `https://testplugin-dx-staging.line-apps.com/api/getSomethingFromYourSide`
- **Request flow**

    1. The browser sends the API request to the plugin domain.
    2. The request is routed to the reverse proxy.
    3. The reverse proxy validates the token/cookies and forwards the request to the partner backend.
    4. The partner backend receives the forwarded request through the reverse proxy.

- **Authentication note**

    - Authentication relies on cookies set on the plugin domain, such as `dx-jwt`.
    - Therefore, partner API calls must be made through the plugin domain rather than calling the partner backend domain directly.

- **Reverse Proxy Handling Limitation for 3xx HTTP Status Codes**

  Please note that our reverse proxy currently does not support any 3xx HTTP status codes other than 304 Not Modified.

  If the upstream service returns a 3xx redirect response, the response will not be forwarded to the frontend as-is. Instead, it will be intercepted by the reverse proxy, and the frontend will ultimately receive a 502 Bad Gateway response.

  The handling rules are as follows:

  - **304** Not Modified: allowed
  - All other 3xx status codes: not supported and will be blocked by the reverse proxy
  - Once blocked, the frontend will receive 502 Bad Gateway instead of the original redirect response

  Important note:

  If your service design or integration flow relies on 3xx redirect behavior, please use an alternative handling approach to avoid unexpected 502 errors on the frontend.

UI Design Guidelines

Language Support Guidelines

- Dynamic Language Switching for zh-TW & en.
- Utilize the language information provided by the SDK to dynamically adjust the admin plugin's language settings.
- Ensure that the plugin interface and content are displayed in the user's preferred language as specified in their profile data.

Responsive Web Design Guidelines

- Ensure that the admin plugin provides an optimal user experience across devices and screen sizes.

Supported Protocols and Standards

Communication Protocols

- **HTTPS (Hypertext Transfer Protocol Secure)**
  - All data exchange between the integration components and external systems must occur over HTTPS.
  - HTTPS ensures that data is encrypted during transmission, providing a secure communication channel.
  - Implement TLS (Transport Layer Security) to establish a secure connection and protect data integrity and confidentiality.

Device & Browser Support

- **Device Support**
  - Mobile devices
  - Tablet (≥ 768px)
  - Desktop
- **Browser Support**
  - Supported: **Chrome 119+**
  - Partially supported: **Safari**

    (May require disabling **“Prevent cross-site tracking,”** and support may vary by partner CMS)

Compliance and Regulatory Requirements

Data Protection and Privacy

- If applicable, ensure compliance with requirements for data protection and privacy for users in Taiwan.
- Implement mechanisms for obtaining user consent, data access, and data deletion requests.

Security Standards

- **Data Encryption**:
  - Ensure that all sensitive data is encrypted both in transit and at rest, using industry-standard encryption protocols.

- **Access Controls**:
  - Implement strict access controls to ensure that only authorized personnel have access to sensitive data and systems.

Compliance Assurance

- To maintain the integrity and security of the integration with the LINE OA Plus TW system, we have implemented several security measures. These measures are crucial to protect both the platform and your applications. Please ensure compliance with the following guidelines:
  - Content Security Policy (CSP) Settings
    - The Content Security Policy (CSP) is designed to control the resources that can be loaded and executed by your application. The following directives will be enforced:
      - **script-src** 'self' https://trusted-scripts.example.com: Scripts can only be loaded from the specified trusted source.
      - **style-src** 'self' 'unsafe-inline' https://trusted-styles.example.com: Styles can be loaded from the specified source, including inline styles.
      - **img-src** 'self' https://images.example.com: Images must be loaded from the specified trusted source.
      - **media-src** 'self': Media must be loaded from same origin.
      - **font-src** 'self' https://trusted-scripts.example.com: font source (.woff, .woff2, .eot) can only be loaded from the specified trusted source.
    - CSP will block all dynamically injected <script> and <link> tags unless they include a valid nonce. This ensures that only pre-approved scripts and styles are executed, enhancing security against injection attacks.
    - All resources must be explicitly defined within the Access Control List (ACL). Resources not listed in the ACL will be blocked by the CSP settings.
  - iFrame Sandbox Configuration
    - To further enhance security, sandboxing is applied with the following allowances:
      - allow-same-origin: Provide operations under the same-origin policy, including the ability to manipulate cookies, localStorage, and IndexedDB within the same origin.
      - allow-scripts: Permits scripts to execute within the sandboxed environment.

LINE OA Plus TW SDK

Frontend SDK URL

ENV	URL
staging	https://vos.line-scdn.net/dx-sdk-staging/v2.0.0/bundle.min.js
production	https://vos.line-scdn.net/dx-sdk/v2.0.0/bundle.min.js

User Management

- **Function**: `getUser()`
  - **Description**: Retrieves a OA user profile associated with the merchant's biz account.

    example

    ```
    <html>
     <script src="https://vos.line-scdn.net/dx-sdk-staging/v2.0.0/bundle.min.js" async/>
     <script>
        dx
         .getUser()
         .then((user) => {
            const merchantId = user.merchantId;
            const language = user.language;
         })
         .catch((err) => {
            console.log("error", err);
         });
     </script>
    </html>
    ```

  - **Syntax**

    ```
    dx.getUser()
    ```

  - **Arguments**

    None

  - **Return Value**

    Returns a **Promise** object.

    When the Promise is resolved, the object containing the user's information is passed.

    - **merchantId** String

      User merchant ID

    - **language** String

      User locale, supports ***zh-TW, en, ja, ko, th, id***

  - **example**

```
{
  "merchantId": "@abc1234",
    "language": "zh-TW"
}
```

- **Error response**

  When the **`Promise`** is rejected, **DxError** object is passed.

Error code	Description	note
TIMEOUT	When the SDK does not receive a response from the OAP within 5000 ms, a timeout occurs.	
EXCEPTION	Some unexpected runtime errors occurred.	

example

```
{
 "code": "TIMEOUT",
 "message": "Failed to get user info"
}
```

- **Function**: `getPluginInfo()`
  - **Description**: Retrieves a user's plugin information about current plugin tab.

example

```
<html>
 <script src="https://vos.line-scdn.net/dx-sdk-staging/v2.0.0/bundle.min.js" async/>
 <script>
      const pluginInfo = dx.getPluginInfo();
      const level = pluginInfo.level;
      const activatedStatus = pluginInfo.activatedStatus;
 </script>
</html>
```

  - **Syntax**

```
dx.getPluginInfo()
```

  - **Arguments**

    None

  - **Return Value**

    Returns a object.

    - **level** String

      User plugin level, supports ***FREE, PREMIUM***

- **activatedStatus** String

  User plugin activatedStatus, supports ***ACTIVATED, ACTIVATION_INITIATED, ACTIVATION_FAILED***

- **example**

```
{
  "level": "FREE",
    "activatedStatus": "ACTIVATED"
}
```

Security

- **Function**: getCSRFToken ( )
  - **Description**: Get a CSRF token for **POST**/**PUT**/**DELETE** request.

    example

```
<html>
 <script src="https://vos.line-scdn.net/dx-sdk-staging/v2.0.0/bundle.min.js" async/>
 <script>
    const csrfToken = dx.security.getCSRFToken();
    const csrfTokenHeaderName = dx.security.getCSRFTokenHeaderName();
    fetch(url, {
            method: 'POST',
            headers: {
              'Content-Type': 'application/json',
                [csrfTokenHeaderName]: csrfToken
            },
            body
      })
 </script>
</html>
```

  - **Syntax**

```
dx.security.getCSRFToken()
```

  - **Arguments**

    None

  - **Return Value**

    Returns a **String** value.

    - **example**

```
iw9lKjgSw4bjCToTFOANs3dVrX0sVKnLhL%2BNkkobh1ZbNxk4w8I%2BrICEP9ScdWCwNyAATE%
2BiY124CQ9xgx5PN2yJrPnXpGy8tVAdhYrbjBA3yE70OitB0tOvC2Ds8rTqFdwhWEQRo0n7Np5JH4ivSsnkdL3wSEC238ZVzw%
2Br
```

- **Function**: getCSRFTokenHeaderName( )
  - **Description**: Get a CSRF header name for setting **POST**/**PUT**/**DELETE** request header.

    example

    ```
    <html>
     <script src="https://vos.line-scdn.net/dx-sdk-staging/v2.0.0/bundle.min.js" async/>
     <script>
        const csrfTokenHeaderName = dx.security.getCSRFTokenHeaderName();
           console.log(csrfTokenHeaderName) // DX-CSRF-TOKEN
     </script>
    </html>
    ```

  - **Syntax**

    ```
    dx.security.getCSRFTokenHeaderName()
    ```

  - **Arguments**

    None

  - **Return Value**

    Returns a **String** value.

    example

    ```
    DX-CSRF-TOKEN
    ```

Open API/Interface Specifications

Prerequisites

Authentication and Authorization

Send a request to the issue Auth Token API endpoint, including all the required parameters and the generated signature.

- **API Request Required Parameters**:
  - **Plugin ID**: Your unique identifier for the plugin.
  - **Merchant ID**: Your unique identifier for the merchant.

- **Timestamp**: The current time in seconds since the epoch, used to ensure the request's freshness.
- **Nonce**: A unique, one-time-use value to prevent replay attacks.
- **Signature**: A cryptographic signature generated using HMAC-SHA256.

- **Short-lived Tokens**:
  - Tokens are valid for 5 minutes, minimizing the risk of misuse if compromised.
  - Tokens are issued with specific scopes that define the permissible actions, ensuring minimal access necessary for your operations.

Endpoint

Environment	Endpoint	Status	Rate Limit
Staging	https://dx-open-api-staging.line-apps.com	Not ready, in progress.	200 RPS
Production	https://dx-open-api.line-apps.com	Not ready, in progress.	800 RPS

Load Test

Do not perform stress testing in the Staging or Production environments.

Only allowed IPs (only in Ipv4 with CIDR format) can access Open API.

APIs

Issue Auth Token

- Description
  - Issues a short-term access token for tech partners to access platform APIs. Implements anti-replay attack protection using timestamp and nonce validation.
- Swagger
  - https://vos.line-scdn.net/oa-plus-dx-swagger-ui-web/index.html#/Authentication/issueToken
- Signature example

```
const openApiSecret = "{YOUR_OPEN_API_SECRET}";
```

```
const bodyRaw = pm.request.body.raw || "{}";
const json = JSON.parse(bodyRaw);

const pluginId = json.pluginId;
const merchantId = json.merchantId;

const timestamp = Math.floor(Date.now() / 1000);

const nonceBytes = crypto.getRandomValues(new Uint8Array(32));
const nonce = Array.from(nonceBytes, b => b.toString(16).padStart(2, "0")).join("");

const data = `${pluginId}:${merchantId}:${timestamp}:${nonce}`;

(async () => {
 const encoder = new TextEncoder();
 const keyBytes = encoder.encode(openApiSecret);

 const key = await crypto.subtle.importKey(
  "raw",
  keyBytes,
  { name: "HMAC", hash: "SHA-256" },
  false,
  ["sign"]
 );

 const dataBytes = encoder.encode(data);
 const sigBuf = await crypto.subtle.sign("HMAC", key, dataBytes);

 const signature = Array.from(new Uint8Array(sigBuf))
  .map(b => b.toString(16).padStart(2, "0"))
  .join("");

 pm.environment.set("timestamp", String(timestamp));
 pm.environment.set("nonce", nonce);
 pm.environment.set("signature", signature);
})();
```

Send Event

- Description
  - Sends event and pushes message.
- Swagger
  - https://vos.line-scdn.net/oa-plus-dx-swagger-ui-web/index.html#/event-controller/sendMessageEvent

Retrieve Subscription

- Description
  - Retrieve the subscription details for a specific plugin associated with given merchants.
- Swagger
  - https://vos.line-scdn.net/oa-plus-dx-swagger-ui-web/index.html#/Plugin%20Subscription/getPluginSubscriptionStatus

Retrieve Subscription V2

- Description
  - **staging only now**
  - Retrieve the subscription and plugin activation details for a specific plugin associated with given merchants.
- Swagger
  - https://vos.line-scdn.net/oa-plus-dx-swagger-ui-web/index.html?urls.primaryName=Staging#/Plugin's%20Subscription%20Status%20and%20Activation%20Details/getPluginSubscriptionStatus

Retrieve Payment

- Description
  - **staging only now**
  - Retrieve the payment details for a specific plugin associated with given payments.
- Swaggerhttps://vos.line-scdn.net/oa-plus-dx-swagger-ui-web/index.html?urls.primaryName=Staging#/Plugin%20Payment/getPluginPaymentDetails

Subscription-Based Access Control

To help ensure consistent behavior across OA Plus and Partner systems after subscription downgrade

- When subscription_status = inactive, tech partners must ensure that ineligible merchants cannot continue using paid features.
- OA Plus provides the "Retrieve Subscription" API , and partners should apply restrictions based on the result.
  - The following items are recommendations, and partners may design stricter or broader limitations depending on service.
  - Front-end (Storefront): It is recommended to suspend or hide the storefront so consumers cannot place new orders.
  - Back-end (CMS): Limit merchant access to *existing orders only*, and disable features that require an active plan.
- API interactions:
  - Avoid requesting new tokens (API will fail)
  - Stop sending Open API events (OA Plus will block them)

Error Handling and Logging

Rate Limiting

- **HTTP Status Code 429**: When a client exceeds the allowed number of requests, the API typically responds with a 429 Too Many Requests status code.

Retry Mechanisms

To enhance the resilience of your integration, we provide specific error codes that indicate when a request can be safely retried. Tech partners should implement the following practices:

- **Error Codes for Retry**:
  - Pay attention to the error codes provided in API responses. Certain codes will indicate that a retry is appropriate.
  - Implement logic to automatically retry requests when these specific error codes are received. This helps manage transient issues such as network timeouts or temporary server unavailability.
- **Exponential Backoff**:
  - Use an exponential backoff strategy for retries to avoid overwhelming the server with repeated requests.
  - Gradually increase the delay between retries to give the system time to recover.
- **Retry Limits**:
  - Set a maximum number of retry attempts to prevent indefinite retries and potential resource exhaustion.

Idempotency Key

To ensure safe API retries and prevent duplicate operations, we have implemented an idempotency mechanism. This allows tech partners to handle network failures gracefully by safely retrying API requests.

- **Client-Generated Event IDs**:
  - Generate a unique event ID for each operation that may require a retry. For example, a createOrder event for an order will have a unique eventId.
    - Suggesting Event ID: merchantId + orderId + eventType + timestamp. (e.g. abc123:12345:Buy:1752122552
  - Include this event ID in the API request to serve as an idempotency key.
- **Idempotency Handling**:

- The server will use the idempotency key to identify and handle repeated requests, ensuring that duplicate operations are not performed.
- This mechanism is particularly useful for operations that modify data, such as creating or updating resources.

Comprehensive Logging

- **Request and Response Logs**: Log all incoming API requests and outgoing responses, including headers, payloads, and status codes, while ensuring that sensitive data is masked or encrypted.
- **Error Logs**: Capture detailed information about any errors or exceptions, including error codes, messages, and stack traces.

Partner Webhook Notifications

Event Type	Description
ACTIVATION_CHANGED	Triggered when a merchant's activation status or subscription level changes.
PAYMENT_SUCCEEDED	Triggered when a payment from a merchant is successfully completed.

Request Format

OA Plus sends an HTTP POST request to the partner's registered webhook URL.

Headers

Header	Value
Content-Type	application/json
X-DX-Signature	sha256={HMAC-SHA256 signature of the raw request body}

Signature Verification

The X-DX-Signature header contains an HMAC-SHA256 signature prefixed with sha256=.

Partners must verify this signature using the Secret of Admin Plugin to confirm the request originates from OA Plus.

```
// Pseudocode for signature verification
String receivedSig = request.getHeader("X-DX-Signature");
String expectedSig = "sha256=" + hmacSha256(rawBody, secret);
```

```
if (!MessageDigest.isEqual(
      expectedSig.getBytes(StandardCharsets.UTF_8),
      receivedSig.getBytes(StandardCharsets.UTF_8)
)) {
   throw new UnauthorizedException("Invalid signature");
}
```

Event Payloads

ACTIVATION_CHANGED

```
{
  "eventType": "ACTIVATION_CHANGED",
  "eventId": "evt-001",
  "pluginId": "testplugin",
  "merchantId": "@merchant123",
  "activatedStatusChanged": {
    "previous": "INACTIVATED",
    "current": "ACTIVATED"
  },
  "pluginLevelChange": {
    "previous": "FREE",
    "current": "PREMIUM"
  },
  "updatedTime": "2026-03-30T16:40:00Z",
  "sentAt": "2026-03-30T16:40:00Z"
}
```

Field	Type	Mandatory	Description
eventType	string	y	Always ACTIVATION_CHANGED
eventId	string	y	Unique identifier for this event
pluginId	string	y	
merchantId	string	y	
activatedStatusChanged.previous	string	n	ACTIVATED or INACTIVATED
activatedStatusChanged.current	string	n	ACTIVATED or INACTIVATED
pluginLevelChange.previous	string	n	FREE or PREMIUM
pluginLevelChange.current	string	n	FREE or PREMIUM
updatedTime	Timestamp	y	ISO 8601 UTC
sentAt	Timestamp	y	ISO 8601 UTC

PAYMENT_SUCCEEDED

```
{
 "eventType": "PAYMENT_SUCCEEDED",
 "eventId": "evt-002",
 "pluginId": "testplugin",
 "merchantId": "@merchant123",
 "paymentOrderId": "pay-001",
 "paymentStatus": "SUCCESS",
 "sentAt": "2026-03-30T16:40:00Z"
}
```

Field	Type	Mandatory	Description
eventType	string	y	Always PAYMENT_SUCCEEDED
eventId	string	y	Unique identifier for this event
pluginId	string	y	
merchantId	string	y	
paymentOrderId	string	y	
paymentStatus	string	y	e.g. SUCCESS
sentAt	Timestamp	y	ISO 8601 UTC

Response Requirements

The partner endpoint must return an HTTP 2xx status code to acknowledge receipt.

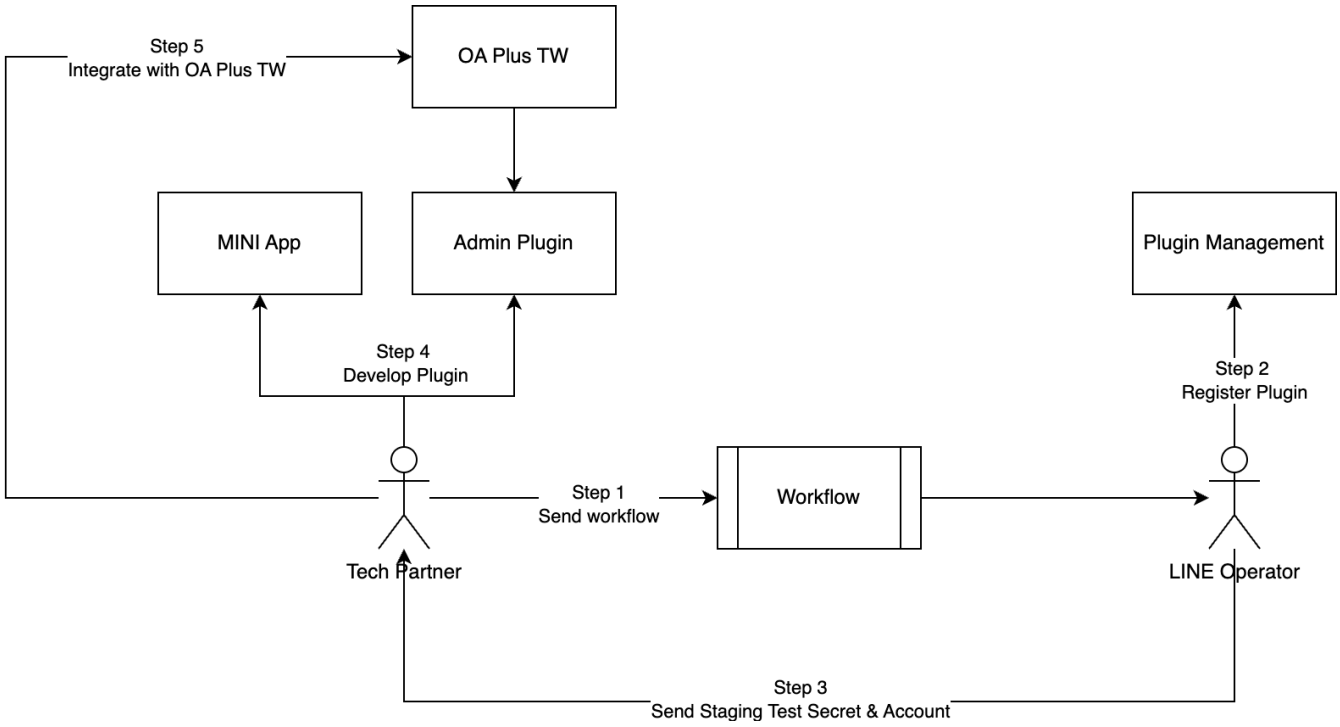
Response Status	Behavior
2xx	Success. No retry.
4xx	Permanent failure. No retry.
5xx or 429	Transient failure. OA Plus will retry up to 3 times with exponential backoff (initial: 1s, multiplier: 2x).

Retry Policy

- Max retry attempts: **3**
- Initial interval: **1000ms**
- Backoff multiplier: **2x**
- Example: 1st retry at 1s, 2nd at 2s, 3rd at 4s

If all retries are exhausted, the event is marked as failed and will not be retried again. Each attempt (including retries) is logged in the webhook_notification_log collection.

Setup and Configuration



Workflow of Step 1

Please ask the form from Stella Hsu.

Integration Info of Step 3

Name	Env	Description
Plugin Id	Staging /Production	The Plugin Id is the unique identifier for the Admin Plugin on our platform.
Secret of Admin Plugin	Staging /Production	To verify requests from the OA Plus TW reverse proxy, the JWT signature must be validated using the secret we provide.
Secret of Issue Auth Token	Staging /Production	To get the access token from Issue Token API, you must use this secret to generate a cryptographic signature using HMAC-SHA256.

OA Plus TW testing account	Staging	Invitation link to become an operator of OA Plus TW testing account

Development and Test

To support partner integration testing, we will provide two testing options based on different OA account tiers.

LINE OA Plus Development Mode

Purpose

This document describes the **Developer Mode** currently supported in OA Plus Extension, including its enablement requirements, usage flow, UI behavior, and limitations. It is intended for internal teams such as PM, engineering, and QA.

Overview

OA Plus Extension now supports Developer Mode.

After we complete the following setup for a specific merchant:

- Register a designated merchant ID to enable Developer Mode
- Configure the test destination URL(s) for that merchant

developers who log in to that merchant's OA Plus admin will see a floating button in the bottom-right corner of the page.

By clicking this button, they can open a control panel where they can:

- Enable or disable Developer Mode
- Choose which destination the current page should be redirected to

This mode is currently supported only in the RC environment, and is designed to make development and testing more convenient.

Use Cases

Developer Mode is mainly intended for the following scenarios:

- Developers need to quickly switch the page redirect destination

- QA or testers need to verify behavior against different destinations in a specific merchant admin panel
- Teams need to perform feature validation, integration testing, or debugging in the RC environment

Enablement Requirements

To use Developer Mode, the following setup must be completed in advance:

1. **Register a specific merchant ID**
   - Only merchants that have been registered and enabled can use Developer Mode.
2. **Configure test destination URL(s)**
   - The destination options used for redirection must be configured beforehand.

Once the above setup is completed, developers logging in to the corresponding merchant admin panel will be able to access Developer Mode.

How to Use

1. Login to OA Plus by specific merchant
   - The developer login to OA Plus by a specific merchant where Developer Mode has already been enabled.
2. Open the floating button
   - A floating button will appear in the bottom-right corner of the page. Click it to open the Developer Mode control panel.
3. Configure Developer Mode
   - In the panel, the developer can:
     - Turn Developer Mode on or off
     - Select which destination the current page should be redirected to
4. Start testing and developing
   - After the configuration is applied, the developer can proceed with redirection and validation based on the selected destination.

UI Description

- Floating Button
  - Location: Bottom-right corner of the page
  - Visibility condition: Visible after a developer logs in to an enabled merchant admin panel

- Developer Panel
  - The panel provides the following capabilities:
    - Enable or disable Developer Mode
    - Select the redirect destination for the current page
  - This design allows developers and testers to switch test destinations quickly without changing the standard production flow.
- Restrictions and Notes
  - RC environment only
    - This mode is currently not supported in the production environment.
  - Merchant ID registration is required
    - Merchants that have not been registered and enabled will not see the related entry point.
  - Destination configuration is required
    - Available destinations must be configured in advance; otherwise, page redirection cannot be performed.

Environment

Environment	URL	Development Mode
Staging	https://oaplus-staging.line.biz/	Yes
Production	https://oaplus.line.biz/	No

Test Accounts

We will provide two OA test accounts for validation:

- **Light**
- **Medium / High**

These accounts are intended to help partners verify different entitlement levels and related behaviors during integration testing in both staging & production environment.

Account Invitation

Once the test accounts are ready, an invitation link will be sent to the partner developer email address.